

Reading Room: Manual AWS EC2 Production-Style Deployment

This guide continues from your existing GitHub repository. Work locally in your own checkout; there is no fork step. You run every provisioning, installation, build, deployment, backup, and reliability command yourself. Editing these files does not create AWS resources or deploy anything.

The application source and README are unchanged. The setup adds private Docker networking, non-root application containers, a single Nginx entry point, readiness checks, persistent storage, rotating logs, secret handling, backup/restore tools, and an operations runbook.

1. Read these assignment adaptations first

Your app is Next.js + Express + MySQL, not the assignment's static/frontend-and-backend-rendered sample. Dynamic `/book/[id]` routes and newly added books need Next.js running on Node. The frontend therefore uses a production Node runtime, not a static-only Nginx runtime. Nginx is still the single public reverse proxy. Application pages go to Next.js, while `/api/` goes to Express.

The source already has a database-backed `/api/books` endpoint. The proxy maps `/health` to that endpoint; no backend source route is added. Screenshot 12 must explain this adaptation. Screenshots 5 and 14 must show the actual Next.js runtime and its HTTP health check. If the assessor requires a static-only frontend and a literal new backend `/health` route, those requirements conflict with preserving your source; seek acceptance of these adaptations rather than claiming exact compliance.

The supplied configuration uses HTTP on port 80, as the assignment requests. It is a hardened single-host assignment deployment, not a complete internet production security posture. Use only safe test accounts/data until you add a domain, TLS, and HTTPS routing. Port 443 and certificate automation are not included. A single EC2 host is not highly available.

2. Understand the final layout

Service	Internal port	Network	Host port
reverse-proxy, non-root Nginx	8080	front-tier	80 only
frontend, Next.js	3000	front-tier	None
backend, Express	3001	front-tier + back-tier	None
database, MySQL 8.4	3306	back-tier, internal	None

MySQL data lives at `/var/lib/mysql` in the named `db_data` volume. By default its Docker name is `reading-room_db_data`. Proxy logs are host files under `APP_DIR/logs/proxy`; backups are host files under `APP_DIR/backups`.

Open `docs/01-architecture.md` for the diagram and `docs/02-env-and-ports.md` for variable names, startup commands, initialization, and checks.

3. Prepare your workstation and existing repository

Install Terraform 1.6+, Ansible, AWS CLI v2, Git, and OpenSSH on your workstation. Docker with Compose v2 is also needed if you want local image builds; Ansible will install Docker on EC2.

Run in your existing checkout:

```
terraform version
ansible-playbook --version
aws --version
git status --short
```

Review and commit the new deployment files to your own repository, then push the commit. The remote VM deploys the exact pushed commit; it does not overlay uncommitted local files.

```
git add infra ansible scripts proxy db docs installation_guide.md \
  docker-compose.yml .env.example .gitignore \
  backend/Dockerfile backend/Dockerfile.baseline backend/.dockerignore \
  frontend/Dockerfile frontend/.dockerignore
git diff --cached --stat
git commit -m "Add private EC2 Compose deployment and operations guide"
git push
```

Inspect staged files before committing. Do not stage `.env`, backups, logs, private keys, Terraform state, or secret payloads. Your README is intentionally excluded.

4. Authenticate to AWS and choose the region at runtime

Open a Bash shell, including on a Mac whose default shell is Zsh:

```
bash
```

Use your configured AWS profile/SSO session; do not place AWS access keys in this repository or on the VM. For an SSO profile, configure/login through AWS CLI if needed:

```
read -r -p "Existing AWS profile name: " AWS_PROFILE
export AWS_PROFILE
aws sso login --profile "$AWS_PROFILE"
aws sts get-caller-identity >/dev/null
read -r -p "Target AWS region: " AWS_REGION
export AWS_REGION
```

For an existing non-SSO profile, skip `aws sso login`. Do not screenshot identity output. The selected region is used for both Terraform and Secrets Manager. Use a supported commercial AWS region with the Ubuntu image and selected instance size.

The provisioning identity needs permissions for EC2/VPC/security groups/key pairs/Elastic IPs, plus IAM role/profile/pass-role operations if you use Secrets Manager. Apply your organization's least-privilege policy. The EC2 instance role created below is separate from your workstation identity.

5. Choose where the secrets will live

Choose one option. Both ultimately materialize a protected mode-0600 `.env` on EC2 because the unchanged application reads environment variables. Secrets never enter Terraform variables/state, Dockerfiles, build arguments, or the Git repository. Docker administrators can inspect container environment values, so restrict SSH and Docker-group access.

Option A: Local protected environment file

```
export SECRET_SOURCE=local
```

After preparing EC2, run `scripts/configure-env.py` on the VM. It prompts for values and hides password/JWT entry. Use distinct random secrets with at least 32 characters from letters, digits, underscores, or hyphens. The initial default database/user names can be accepted or supplied at runtime. This option does not create an AWS secret or EC2

secret-reading role.

Option B: AWS Secrets Manager

AWS Secrets Manager is the managed secret store used here. It is different from the third-party `aws-vault` credential helper. Follow these steps before planning the infrastructure:

1. In AWS Console, select the same region you exported as `AWS_REGION`.
2. Open Secrets Manager and choose **Store a new secret**.
3. Select **Other type of secret** and enter these five key/value pairs: `MYSQL_DATABASE`, `MYSQL_USER`, `MYSQL_ROOT_PASSWORD`, `MYSQL_PASSWORD`, `JWT_SECRET`.
4. Choose your application database/user names. Use three different randomly generated password/JWT values, at least 32 safe characters each. For example, generate each separately with `openssl rand -hex 32` in a private, unrecorded terminal. Never include that output in screenshots.
5. Use the default AWS-managed `aws/secretsmanager` encryption key. The included IAM role permits only `GetSecretValue` on this one secret. A custom KMS key needs additional narrowly scoped `kms:Decrypt` permission and a matching key policy; it is not configured automatically.
6. Enter a secret name, such as your chosen application/environment path. Do not enable automatic rotation for this generic JSON secret without a rotation implementation that also changes the MySQL users.
7. Store the secret, open its detail page, and copy its ARN privately. Do not capture its values or account ID in evidence.
8. Export these runtime inputs in your workstation terminal:

```
export SECRET_SOURCE=aws
read -r -p "Secrets Manager ARN (private setup, not evidence): " SECRET_ARN
export SECRET_ARN
```

Terraform will attach a role to the EC2 instance with permission to retrieve this existing secret only. The host AWS CLI uses that instance role; no access keys are installed on EC2. The secret is read when you explicitly run the configure phase, not continuously. A secret version change does not automatically change initialized MySQL passwords.

6. Prepare SSH keys and collect runtime inputs

Use an existing dedicated key pair, or create a new one with a unique filename. Do not overwrite another key. Keep the private key on your workstation; Terraform reads only the matching public key.

```
ssh-keygen -t ed25519 -f "$HOME/.ssh/reading-room" -C "reading-room-ec2"
chmod 600 "$HOME/.ssh/reading-room"
source scripts/runtime-inputs.sh
```

The prompt collects and exports the region, x86_64 instance type, disk size, your public IPv4 /32, absolute private/public key paths, your own repository URL, the exact pushed commit, EC2 deployment directory, secret source/ARN, and full name. Defaults are offered for instance size (`t3.medium`), disk (30 GiB), checkout commit, repo URL, and directory. These are runtime defaults, not secrets or a fixed region.

Find your current public IPv4 through your network provider or an IP lookup. SSH must be your IPv4 followed by /32, never 0.0.0.0/0. Use absolute SSH paths, not a literal `~` entered at the prompt. The included Ubuntu AMI is x86_64; do not select an ARM/Graviton instance.

Environment exports apply to the current Bash terminal. Keep that terminal open for the following phases. For another terminal, export the same inputs again. Do not run `env`, `printenv`, or `set -x` for evidence. Leave any old `infra/terraform.tfvars` unused; the helper passes runtime values explicitly with higher precedence.

7. Plan and manually apply the infrastructure

```
bash scripts/deploy.sh plan
```

Review the plan. Expected resources are a VPC/public subnet/route/Internet gateway, restricted security group, key pair, one encrypted-disk Ubuntu EC2 instance, an Elastic IP, and optionally the one-secret IAM role/profile. Expected inbound ports: SSH 22 only from your /32, and HTTP 80 publicly. No rules for 3000, 3001, or 3306.

If continuing from an existing EC2 deployment, inspect replacement/deletion actions carefully: replacing its root disk can destroy local database data. Back up existing data before applying any replacement. This setup does not promise AWS free-tier eligibility.

When you decide to apply the reviewed saved plan:

```
bash scripts/deploy.sh apply
```

The helper asks you to type `apply` because Terraform saved-plan application has no confirmation prompt of its own. It applies infrastructure only. It does not install Docker or start containers. Generate a new plan after changing runtime inputs or infrastructure files; do not apply a stale plan.

Read the outputs into runtime variables:

```
export APP_IP="$(terraform -chdir=infra output -raw public_ip)"
export INSTANCE_ID="$(terraform -chdir=infra output -raw instance_id)"
export PUBLIC_URL="http://$APP_IP"
```

8. Verify the SSH host key before preparation

Use the EC2 Console **Get system log** or another trusted AWS access method to find the instance's ED25519 SSH host-key fingerprint. Then connect from your workstation:

```
ssh -i "$SSH_KEY" ubuntu@$APP_IP true
```

Compare the displayed fingerprint against the trusted instance fingerprint before accepting it. This creates your known-hosts entry. The deployment script uses `StrictHostKeyChecking=yes`; it does not bypass host verification.

For a public repository, proceed to step 9. For a private repository, use its SSH URL and a read-only GitHub deploy key:

1. Connect to the VM and generate a dedicated GitHub SSH key at `/home/ubuntu/.ssh/reading-room-github`. Do not reuse the EC2 login key.
2. Add only that key's public half under your own repository's **Settings > Deploy keys**, with write access disabled.
3. Verify GitHub's SSH host fingerprint against [GitHub's published fingerprints](#), then establish the VM's GitHub known-host entry. An SSH authentication test may exit with status 1 even when GitHub confirms successful authentication because it has no interactive shell.
4. On the workstation, export `APP_REPO` as your `git@github.com:OWNER/REPO.git` URL and `GITHUB_DEPLOY_KEY=/home/ubuntu/.ssh/reading-room-github`. This is a path on the VM, not a private key value.

Never embed a GitHub token in the repo URL.

9. Manually prepare the VM

```
bash scripts/deploy.sh prepare
```

This user-invoked phase installs Docker/Compose/Git/Python/AWS CLI, checks out your own repository at `APP_REF`, creates backup/log directories, and installs proxy log rotation. It neither creates application secrets nor starts the stack. Local changes are not force-discarded; a dirty remote checkout can stop preparation for review.

Reconnect to the VM after its Docker group membership changes:

```
ssh -i "$SSH_KEY" ubuntu@"$APP_IP"
```

In the VM shell, enter the chosen APP_DIR (default /opt/reading-room) and set runtime values again; workstation exports do not pass automatically over SSH:

```
bash
read -r -p "Deployment directory: " APP_DIR
export APP_DIR
cd "$APP_DIR"
read -r -p "Full name for evidence: " FULL_NAME
export FULL_NAME
read -r -p "Public origin, e.g. http://your-EC2-IP: " PUBLIC_URL
export PUBLIC_URL
read -r -p "Secrets source, local or aws: " SECRET_SOURCE
export SECRET_SOURCE
export APP_IMAGE_TAG="$(git rev-parse HEAD)"
```

Do not make the Docker socket world-writable. Docker-group membership grants extensive host privileges.

10. Preserve an existing database or select a new volume

For a new installation, set the default named volume:

```
export DB_VOLUME_NAME=reading-room_db_data
```

For an upgrade from this repo's older Compose setup, first check the old volume's existence without deleting it:

```
docker volume inspect reading-room_mysql_data --format '{{.Name}}'
export DB_VOLUME_NAME=reading-room_mysql_data
```

Use the existing database/user/password values. Do not generate different MySQL passwords and expect an initialized volume to adopt them. Back up the old stack before the transition. Before starting the new database service against that same volume, stop the old stack using its OLD Compose file; never run two MySQL instances against one data directory. Record the old image version and configuration for rollback. Do not run `down -v`.

If you exported DB_VOLUME_NAME for an upgrade on the VM, also export it with that same value in your workstation terminal before the configure phase. A newly written `.env` preserves it for later runs.

11. Materialize the protected environment file

On the VM, for local mode:

```
python3 scripts/configure-env.py
```

It prompts for missing values, stores each input in its process environment, and writes `.env` at mode 0600 without printing secrets. For existing local `.env` values, it reuses them and refreshes non-secret settings. Password/JWT inputs are hidden.

For Secrets Manager mode, provide the region and ARN on the VM first:

```
read -r -p "AWS region: " AWS_REGION
export AWS_REGION
read -r -p "Existing secret ARN (private setup): " SECRET_ARN
export SECRET_ARN
python3 scripts/configure-env.py
```

The instance role retrieves the secret. No `aws` configure or AWS access keys are needed on EC2. If access fails, check the instance profile, region, secret ARN, and IAM propagation. Do not screenshot ARN/account details.

The script rejects changed existing secret/database identity values unless you explicitly authorize a planned rotation with `ALLOW_SECRET_UPDATE=yes`. Never use that override to bypass MySQL password synchronization. See the runbook first.

Check file permissions and Git exclusion without printing contents:

```
stat -c '%a %n' .env
git check-ignore .env
git ls-files .env
docker compose config --quiet
```

The mode should be 600; `.env` should be ignored and absent from tracked-file output. Do not run `cat .env`, `docker inspect` without a restricted format, or unqualified `docker compose config` in screenshots.

12. Build images and record the baseline comparison

Run on the VM. These are manual commands; no service starts yet:

```
printf 'Full Name: %s\n' "$FULL_NAME"
cat backend/Dockerfile frontend/Dockerfile
cat backend/.dockerignore frontend/.dockerignore
docker build --pull -f backend/Dockerfile.baseline -t reading-room-backend:baseline backend
docker compose build --pull
docker image inspect reading-room-backend:baseline --format '{{.Size}}'
docker image inspect "reading-room-backend:$APP_IMAGE_TAG" --format '{{.Size}}'
docker image inspect "reading-room-frontend:$APP_IMAGE_TAG" --format '{{.Size}}'
```

Record actual bytes/MiB and compute $(\text{baseline} - \text{optimized}) / \text{baseline} * 100$. Do not invent an image-size result. Explain that copying dependency manifests before source improves build caching and that non-root containers reduce process privileges. The baseline uses a full Debian Node image; the optimized runtime uses Alpine and production dependencies only. The frontend's final image includes built output/runtime dependencies, not frontend source or build tooling.

Scan the built images using your approved image scanner before a real release. The previous npm audit result does not cover OS packages. `--pull` refreshes mutable base tags; use tested digest-pinned `NODE_IMAGE`, `MYSQL_IMAGE`, and `NGINX_IMAGE` values for a reproducible release. The Nginx version must support upstream `resolve` (1.27.3+).

13. Validate and start the stack yourself

Either run directly on the VM:

```
docker compose config --quiet
docker compose up -d --build --wait --wait-timeout 600
```

Or return to your workstation's configured Bash terminal and invoke:

```
bash scripts/deploy.sh configure
```

Choose one method for this deployment. The configure phase verifies the prepared commit, retrieves AWS secrets or reuses the existing local env file, validates Compose quietly, starts the stack, and checks the page and health endpoint. It does not run Terraform or implicitly prepare a new commit.

On the VM, verify the running backend UID and image startup user:

```
printf 'Full Name: %s\n' "$FULL_NAME"
docker compose ps
docker compose exec -T backend id
docker image inspect "reading-room-backend:$APP_IMAGE_TAG" --format '{{.Config.User}}'
```

The backend should run as node, UID 1000. Only reverse-proxy should show a published host port. Frontend/backend/database may show internal ports, but must not show host mappings.

14. Verify routing, health, logs, and browser functionality

On the VM:

```
printf 'Full Name: %s\n' "$FULL_NAME"
bash scripts/verify.sh
curl --fail --silent --show-error "$PUBLIC_URL/health"
docker compose logs --tail=50 backend
sudo tail -n 10 logs/proxy/access.log
sudo tail -n 10 logs/proxy/error.log
docker compose restart reverse-proxy
docker compose up -d --wait --wait-timeout 600
bash scripts/verify.sh
sudo ls -l logs/proxy
```

The verification script requests a page, /api/books, a real built /_next/static/ asset, /health, and an intentional missing page expected to return 404. The public health body is the existing books response, not a new health JSON format. Empty error logs are normal when no errors occur.

Open PUBLIC_URL in your browser. Test registration/login using safe test credentials, adding a book, 1-5 star reviews, editing/deleting your own review, searching/sorting, Discover navigation, and My favourites. Favourites remain browser-local and do not sync between devices. Use DevTools to confirm browser API requests are same-origin /api/...; never screenshot authorization headers or tokens.

Moving from the old port-3000 URL to the port-80 origin gives the browser a different local-storage namespace. Existing favourites stay at the old origin; MySQL account/book/review data is preserved through the selected volume. Browser-local favourites are not included in database backups.

15. Perform the backup, controlled restore, and persistence drill

Use a test environment or scheduled maintenance window. A full restore replaces data to the backup point. Stop application writers before taking the drill backup so later user changes are not lost.

```
printf 'Full Name: %s\n' "$FULL_NAME"
docker compose ps
export DRILL_ID="$(date -u +%Y%m%dT%H%M%S)"
bash scripts/db-drill.sh create
bash scripts/db-drill.sh show
docker compose stop reverse-proxy frontend backend
bash scripts/backup.sh
ls -lh backups/*.sql.gz
```

Record the filename printed by the backup script. Select that exact backup interactively, without choosing an unrelated older backup:

```
read -r -p "Exact drill backup path: " BACKUP_FILE
export BACKUP_FILE
bash scripts/db-drill.sh delete
bash scripts/db-drill.sh show
export CONFIRM_RESTORE=restore
bash scripts/restore.sh "$BACKUP_FILE"
unset CONFIRM_RESTORE
bash scripts/db-drill.sh show
docker compose up -d --wait --wait-timeout 600
bash scripts/verify.sh
```

Deletion targets only the dedicated test book and refuses books with reviews. Show the missing row before restore and restored row afterward. If deletion reports zero, review the drill record rather than broadening the deletion query.

Verify the physical database mount using a restricted inspect format:

```
docker inspect "$(docker compose ps -q database)" \
  --format '{{range .Mounts}}{{println .Name .Destination}}{{end}}'
docker compose down
docker compose up -d --wait --wait-timeout 600
bash scripts/db-drill.sh show
bash scripts/verify.sh
```

Never add `-v` to this cycle. Fill docs/05-persistence-and-backup.md with actual backup/drill observations.

For scheduled local backups, edit the VM user's crontab manually with `crontab -e`. For the default path, a daily example is:

```
0 2 * * * /usr/bin/bash /opt/reading-room/scripts/backup.sh >> /opt/reading-room/backups/backup-job.log 2>&1
```

Replace the path if you chose a different `APP_DIR`. Keep seven daily local backups and at least thirty daily encrypted off-host copies as a starting recommendation; implement retention only after reviewing which files are safe to remove. No scheduler/retention job is installed automatically. Configure a private encrypted S3 bucket, block public access, grant a dedicated backup identity access to its backup prefix, upload copies, and test restoration from S3. The supplied EC2 secret-reading role does not grant S3 access.

16. Perform controlled reliability tests

Read docs/07-operations-runbook.md. Confirm all services are healthy first. These tests cause temporary outages; run them in a maintenance/test window.

Backend test:

```
printf 'Full Name: %s\n' "$FULL_NAME"
bash scripts/verify.sh
docker compose stop backend
curl --silent --show-error --max-time 40 -o /dev/null -w 'Backend outage HTTP %{http_code}\n' "$PUBLIC_URL/api/books"
docker compose start backend
docker compose up -d --wait --wait-timeout 600
bash scripts/verify.sh
```

The proxy should return the configured 503 while the backend is stopped. Database test:

```
docker compose stop database
curl --silent --show-error --max-time 40 -o /dev/null -w 'Database outage HTTP %{http_code}\n' "$PUBLIC_URL/api/books"
docker compose start database
docker compose up -d --wait --wait-timeout 600
bash scripts/verify.sh
```

The database-dependent API request should fail during the outage. Confirm reconnection and healthy responses afterward. If reconnection does not complete, inspect safe logs and restart backend only after the database is healthy; record that fallback. Health checks do not automatically restart unhealthy containers. Do not delete a volume, table, or real record during reliability tests.

Record actual times/statuses/actions in the runbook's reliability table. Do not claim a successful test before observing it.

17. Updates and rollback

Push your next configuration/source commit to your existing repository. On the workstation, export the full new SHA as `APP_REF` and `APP_IMAGE_TAG`, then run prepare and configure as separate manual phases. Back up first and verify afterward. Retain the previous commit, image tags/digests, and backup filenames.

For a code rollback, set those variables to the previously tested full SHA and repeat prepare/configure. Check schema compatibility and avoid automatic database restore. See the runbook for startup/shutdown, restarts, logs, backup/restore, secret rotation, and post-incident checks.

CI/CD task 8 is optional and intentionally not added: this workflow is operator-driven. No push-triggered cloud deployment runs on your behalf.

18. Screenshot and submission checklist

Before each terminal capture, run `printf 'Full Name: %s\n' "$FULL_NAME"`. Add your full name as a clear caption beneath each browser/console screenshot and in the architecture diagram. Never show real environment contents, credentials, tokens, private keys, account IDs, or secret values. Capture tasks sequentially and use actual VM results.

Screenshot	Capture
1	Existing repository structure on the VM, without a fork claim
2	Architecture diagram and full-name caption
3	docs/02-env-and-ports.md, names only
4	Backend dependency/runtime Dockerfile stages
5	Actual frontend Dockerfile and documented Next.js adaptation
6	Both application .dockerignore files
7	Successful baseline/optimized/frontend builds and real sizes
8	Running backend non-root UID
9-10	Four Compose services, both networks, volume
11	Successful quiet Compose validation
12	Proxy /health mapping plus existing /api/books query code; explain adaptation
13-14	Four health checks and healthy dependency conditions
15-16	Healthy service status and successful public health request
17	docs/03-healthchecks-and-depends-on.md
18-19	Nginx routes and proxy-only port publication
20	Page/API/real static asset/health verification output
21-22	Browser app and docs/04-proxy-routing-and-cors.md
23	Restricted-format database volume/mount output
24-25	Dedicated test record before drill; successful host backup
26-27	Only drill record missing, then restored
28-29	Same data after down/up; persistence/backup document

Screenshot	Capture
30-31	JSON log config, bind mount, retained logs after restart, backend logs
32	EC2 IP and inbound 22-/32 + 80 rules, account details concealed
33-34	Healthy stack, public response, no internal host ports, browser app
35-36	Backend/database outage responses and actual recovery

Include actual image sizes and reduction, log format/path notes, chosen region/provider/public URL/firewall notes, completed backup results, and the 1-2 page operations/reliability record. Templates with pending results are not execution evidence.

Reference documentation

- Docker Compose private networking
- Docker installation on Ubuntu
- AWS Secrets Manager retrieval and permissions
- Nginx upstream resolution
- Next.js static export limitations